

PocketSmart AI

AI Budget Planner & Recommendation Assistant

Project Documentation

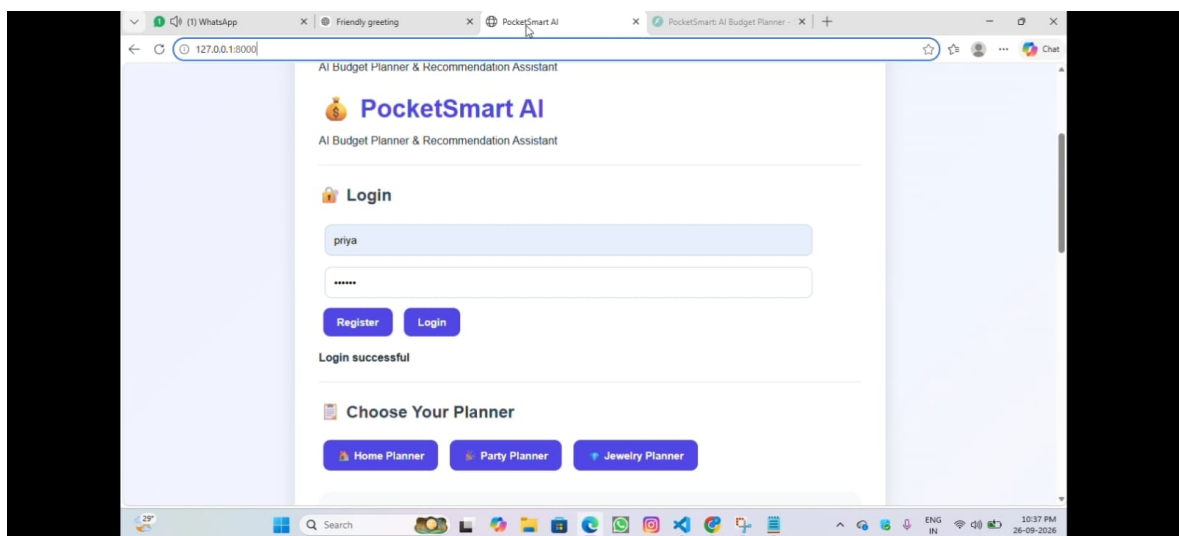


Figure 1: PocketSmart AI login screen and planner selection

1. Overview

PocketSmart AI is a web application that helps users plan budgets and receive smart recommendations for three everyday needs: home interiors, party planning, and jewelry purchases. Users register and log in, choose a planner, describe their requirement and budget, and receive a recommendation. Every recommendation is saved to the user's history and can be reviewed from the dashboard.

The backend is built with Python and FastAPI. AI recommendations are designed to be generated by Google Gemini through a dedicated service layer.

2. Features

- **User authentication** - register and login with session handling.
- **Home Planner** - budget-friendly interior and living-room ideas.
- **Party Planner** - recommendations for organizing a party within budget.
- **Jewelry Planner** - jewelry suggestions based on budget and occasion.
- **Recommendation history** - past requests and results saved per user.
- **User dashboard** - a personal view of the user's activity.
- **Interactive API docs** - Swagger UI available at /docs.

3. Tech Stack

Layer	Technology
Language	Python
Backend framework	FastAPI
Server	Uvicorn
AI engine	Google Gemini API (via gemini_service.py)
Frontend	HTML, CSS, JavaScript (served at 127.0.0.1:8000)
API documentation	Swagger UI (auto-generated by FastAPI)
Editor	Visual Studio Code

4. Project Structure

```
Pocket Smart AI/  
|-- app/  
|   |-- main.py           # FastAPI application entry point  
|   |-- models.py        # Data models  
|   |-- routes/  
|       |-- auth.py       # Register / login  
|       |-- home.py       # Home planner endpoint  
|       |-- party.py      # Party planner endpoint  
|       |-- jewelry.py    # Jewelry planner endpoint  
|       |-- history.py    # Save / fetch user history  
|       |-- recommendations.py # Recommendation details
```

```
| | |-- session.py          # Session info and data
| | |-- startup.py         # Startup route
| |-- services/
| | |-- auth_service.py    # Authentication logic
| | |-- gemini_service.py  # Gemini AI integration
| | |-- history_service.py # History storage logic
| |-- recommendation_service.py # Recommendation logic
```

5. API Endpoints

All endpoints are visible and testable in the Swagger UI at <http://127.0.0.1:8000/docs>.

Method	Endpoint	Description
POST	/generate-home	Generate a home interior recommendation
POST	/generate-party	Generate a party planning recommendation
POST	/generate-jewelry	Generate a jewelry recommendation
POST	/register	Register a new user
POST	/login	Log in an existing user
GET	/session-info	Get current session information
GET	/session-data	Get current session data
GET	/recommendations-details	Get recommendation details
POST	/save-history	Save a recommendation to user history
GET	/history	Fetch the user's recommendation history
GET	/startup	Startup / health route

Example request

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/generate-home?details=Budget 50000, simple living room' \  
  -H 'accept: */*' -d ''
```

Example response

```
{  
  "category": "Home Interior",  
  "recommendation": "PocketSmart AI Recommendation ..."  
}
```

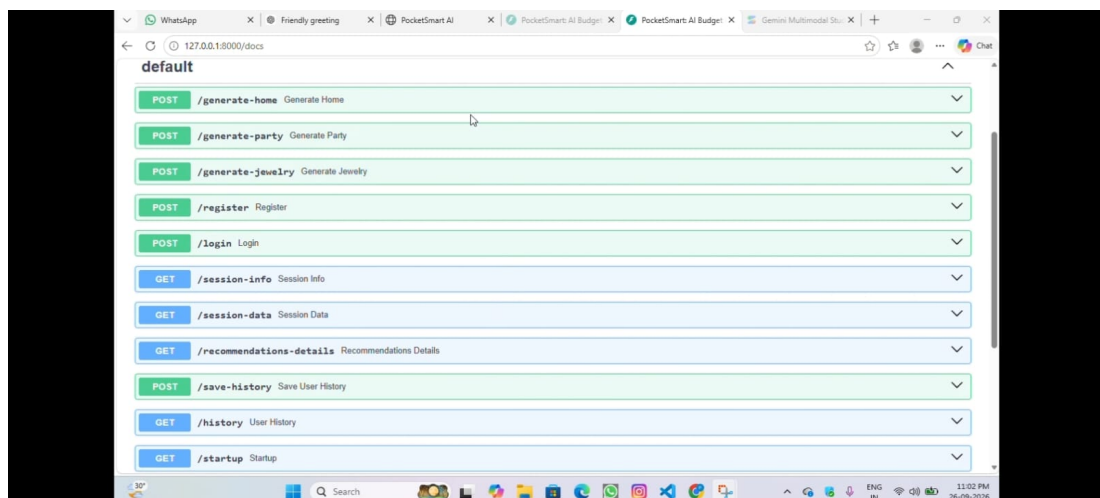


Figure 2: Swagger UI listing all PocketSmart AI endpoints

6. Application Screenshots

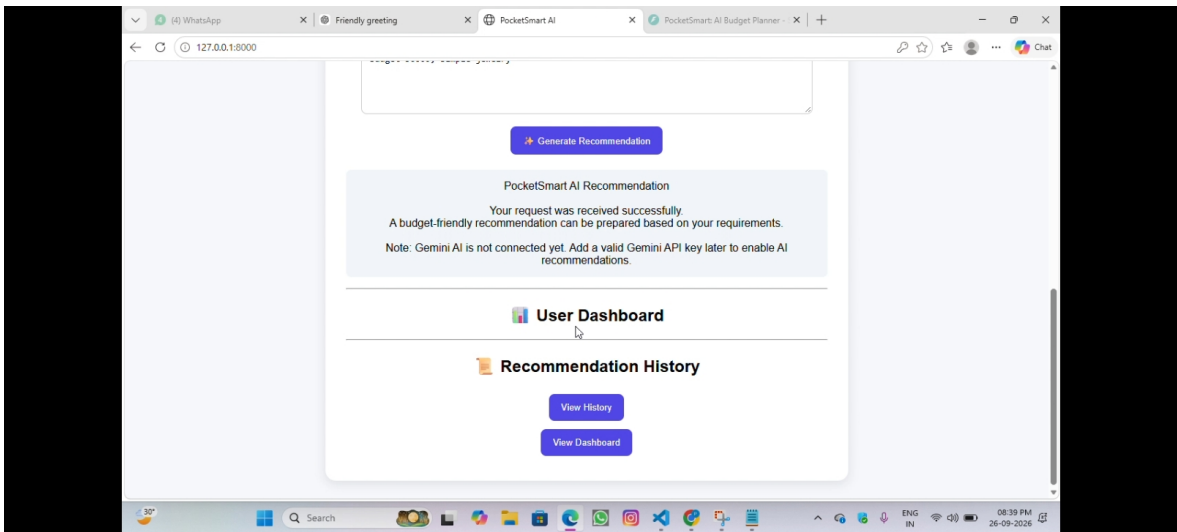


Figure 3: Recommendation output and user dashboard section

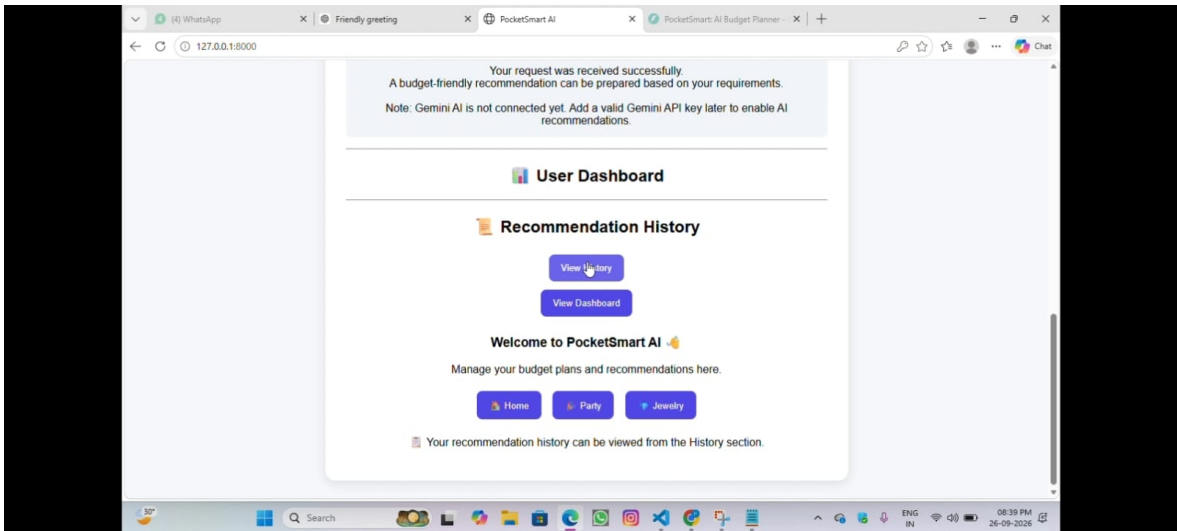


Figure 4: Recommendation history and quick access to Home, Party and Jewelry planners

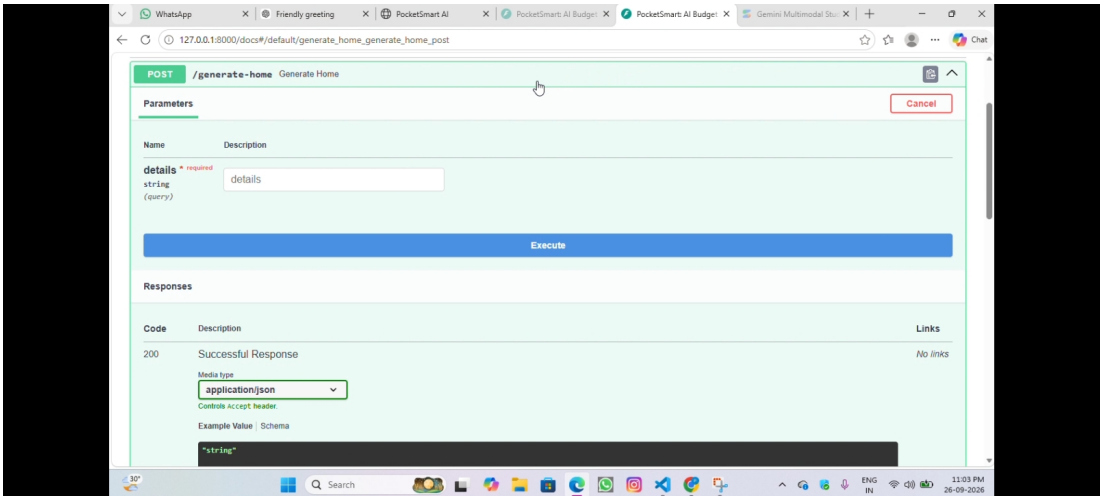
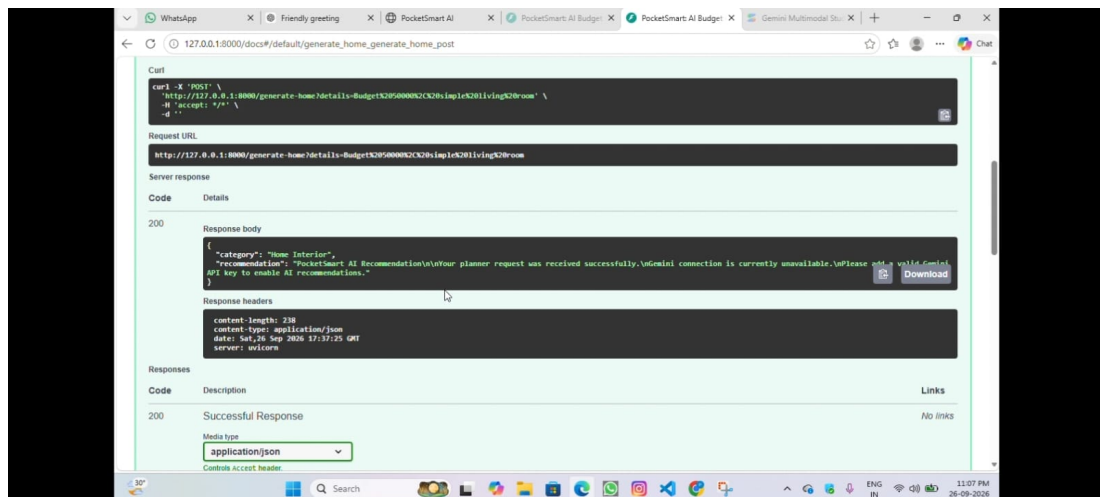


Figure 5: Testing the /generate-home endpoint in Swagger UI



7. Getting Started

```
# 1. Clone the repository
git clone <your-repo-url>
cd "Pocket Smart AI"
```

```
# 3. Install dependencies
pip install fastapi uvicorn google-generativeai
```

Then open `http://127.0.0.1:8000` for the app or `http://127.0.0.1:8000/docs` for the API docs.

Add a valid Gemini API key in the Gemini service configuration. Never commit your API key to GitHub - store it in an environment variable or a .env file and add that file to .gitignore.

- Authentication, planners, history and dashboard are working.
- Recommendations currently return a friendly placeholder message, because the Gemini API key has not been added yet. Once a valid key is configured, responses will be AI-generated.

- Connect Gemini to produce real, personalized budget breakdowns.
- Add charts and spending insights to the user dashboard.
- Move from session storage to a database (SQLite / PostgreSQL) with hashed passwords.

- Add more planners (travel, wedding, gadgets) and export results as PDF.
- Deploy the app online (Render, Railway or similar).

PocketSmart AI - plan smarter, spend wiser.